

Validate the clock before using the data

Daily learning note | 27 September 2026 | Q005

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a strict ingestion boundary reject malformed observations and invalid local timestamps without silently choosing a clock interpretation? The hypothesis is that explicit offsets, verified against one declared source zone, distinguish repeated times and reject missing times before UTC selection.

Yesterday's Q005a made selection chronological once inputs identify valid instants. It did not establish whether a source's local clock label exists. Q005 completes this next boundary while lawful market-data access remains unverified. Today's work is independent engineering research, not an empirical alpha test.

Beginner explanation and definitions

Ingestion means admitting external records into a defined internal representation. A parser can read a string successfully even when its interpretation is unsuitable. A source saying London 01:30 during the spring gap is like naming a nonexistent address: adding an offset does not make that address valid.

Wall time: calendar fields shown on a local clock. **Offset:** local time minus UTC. **Named zone:** a location's historical clock rules. **Fold:** a repeated local interval. **Gap:** an interval skipped by a forward clock change. **Round trip:** convert to UTC and back to check the original interpretation. **Version key:** asset, feature, observation instant and publication instant, used to detect duplicates.

The policy requires explicit offsets for all timestamps. For an ambiguous label, either valid offset is acceptable; the source must supply it. Missing offsets are rejected even on ordinary dates. The code checks internal consistency, not whether the source really published a value at its claimed time.

Data, metrics and predeclared protocol

All values are invented. The main fixture is London on 25 October 2026, with gap controls on 29 March. Tests also use New York's 8 March/1 November transitions and Lord Howe's 5 April/4 October transitions. These dates are clock-rule fixtures, including future dates, not fabricated records of research activity.

Acceptance: two valid fold versions map to hand-calculated UTC instants; all eight invalid batches fail; duplicates fail after normalization; valid revisions retain order-independent selection; and malformed schemas, identifiers, numeric values and timestamp formats cannot enter the batch. No partial result returns on failure. Two independent experiment executions must exactly match the archived JSON.

Mathematics and worked example

Write a source timestamp as wall fields w and explicit offset u . Its proposed UTC instant is $t = w - u$. Let $R_z(t)$ convert t into the declared zone z . Admission requires $w(R_z(t)) = w$ and $offset(R_z(t)) = u$. Checking only t is insufficient: any fixed offset may define an instant even for a missing local label.

For an observation, let o be its observation instant and a its availability instant. Require $a \geq o$. A version key is $K = (\text{asset}, \text{feature}, o, a)$. Two rows with equal K are rejected, even if their values agree. A revision keeps o and changes a . At decision d , the existing selector considers only $a \leq d$, with any configured processing latency and maximum age still applied in elapsed UTC time.

Synthetic London input	UTC interpretation	Admission
25 Oct 01:30, +01:00	25 Oct 00:30	Accept first occurrence
25 Oct 01:30, +00:00	25 Oct 01:30	Accept second occurrence
29 Mar 01:30, +00:00	29 Mar 01:30, returns as 02:30	Reject gap
29 Mar 01:30, +01:00	29 Mar 00:30, returns as 00:30	Reject gap

The first version is observed and published at the first autumn 01:30, with value 1. A revision arrives at the second 01:30 with value 2. Decisions at 00:30 UTC and 01:30 UTC must therefore return 1 and 2. The two identical wall labels are one hour apart. A duplicate written with Z instead of +00:00 remains the same version; spelling changes do not create new information.

Not every clock change is an hour. The Lord Howe fold fixture maps 01:45 at +11:00 to 14:45 UTC on the previous date, and 01:45 at +10:30 to 15:15 UTC. Our validation makes no one-hour assumption.

Primary sources and design choices

PEP 495 explains fold markers, missing times and the absence of strict invalid-time checks in ordinary constructors. It motivates validating the source-clock interpretation separately from construction.
<https://peps.python.org/pep-0495/>

Python's zoneinfo documentation describes IANA zone data and offset-aware conversion; datetime documents ISO parsing and conversion between zones. Our grammar is intentionally narrower than fromisoformat. <https://docs.python.org/3/library/zoneinfo.html>
<https://docs.python.org/3/library/datetime.html>

These official sources were consulted on 27 September 2026. Requiring explicit offsets, one declared zone per batch and rejection of duplicate versions are project policies, not universal source formats.

Implementation and reproduction

The reusable adapter is `src/quant_research/data/ingestion.py`. It accepts mappings with exactly five fields: `asset`, `feature`, `observed_at`, `available_at` and `value`. Identifiers must be printable nonempty strings without edge whitespace. Values are finite built-in integers/floats, excluding booleans. Each timestamp has seconds, at most six fractional digits and Z or an explicit `+/-HH:MM` offset. Unknown `-00:00` offsets, naive labels, leap seconds and extra precision fail; none are silently repaired.

A single IANA `source_zone` applies to both fields of every row. London summer timestamps expressed in UTC should enter a UTC-declared batch; a London-declared batch requires its local labels and correct offsets. Seasonal offset changes within one named zone are valid. This is a declared representation contract, not a prohibition on converting between zones before ingestion with preserved provenance.

The adapter converts and validates every record before returning an immutable tuple in input order. Errors identify the row and rule without echoing source values. It does not mutate mappings. A failing iterator is consumed up to its failure; there is no promise of rolling back external side effects.

Run `make ingestion-experiment` for the diagnostic and `make check` for the complete checks. The experiment accepts `--output PATH` for scratch replays. Run `make manifest-experiment` for prior provenance evidence; run `make note NOTE_DATE=2026-09-27` to rebuild this note. Runtime dependencies are unchanged.

Observed results and validation

Check	Observed result
Valid fold versions	2 accepted; availability 00:30 and 01:30 UTC
Decision values	1 at first instant; 2 at second instant
Invalid experiment controls	All 8 rejected
New ingestion tests	90 passed; 24 valid input permutations checked
Full Python suite	257 passed
Code quality	Ruff; 30 formatted files; mypy on 16 source files passed
Build and dependencies	Warning-clean C++ golden-output check; pip consistency passed
Reproducibility	Two subprocess outputs matched each other and archived JSON
Prior provenance audit	Four source hashes still match; changed-byte control rejected

An initial strict type check found that runtime type validation did not narrow an object for mypy; an explicit type cast after that check resolves it without coercing invalid input. The first full run found two overlong source lines; they were wrapped and the full check passed. Restricted fetch failed DNS; approved fetch succeeded. The authenticated issue read returned zero open issues. No code-coverage percentage or remote CI outcome is inferred from these local checks.

Limitations and interview explanation

Structural validity does not prove data entitlement, a truthful publication clock, vintage completeness, correct security identifiers or trading-session membership. Direct Observation construction still assumes upstream validation; this is an explicit adapter, not a repository-wide file ingestion service. Raw-file parsing, persistent cross-batch uniqueness and calendar validation remain separate responsibilities.

Duplicate detection covers one batch. The selector retains its duplicate check when batches are combined. Raw source text, zone metadata and timezone-database provenance must be retained upstream because the returned observations store UTC only. Available system/IANA rules are not a pinned embedded database; expected-offset fixtures expose relevant changes but do not certify all historical dates.

Floating-point conversion can round large integers or decimals; this interface is for scalar research features, not exact monetary accounting. There is no data download, subscription, new runtime dependency, measured trading result or performance claim. Q004a still blocks empirical research until access is verified.

For an interview, explain the difference between parsing, validating and authenticating a timestamp. Draw both autumn instants and a missing spring label. Show the UTC round-trip equations, explain why an explicit offset resolves a fold but cannot rescue a gap, and finish with the duplicate-version invariant.

Next work and required input

Q006 is ready to define market-calendar and decision-time rules. Q007 belongs to today's 16:00 weekly synthesis, which should replan from actual evidence without repeating this morning's task. C002 remains the next Wednesday C++ step on 30 September. Original target dates are preserved for weekly replanning. No user decision is needed for the next synthetic engineering step. Source entitlement or budget input is needed before Q004a acquisition. Personal review remains separate from permission to continue.

Check your understanding

1. Why is a successfully parsed offset timestamp insufficient to validate a London source clock?
2. Why accept two offsets for an autumn fold but reject both offsets for a spring gap?
3. Are equivalent Z and +00:00 spellings different versions? What creates a revision?
4. Can a failed batch return its valid prefix? What happens to a supplied iterator?
5. Does this adapter prove point-in-time market-data authenticity or exchange-calendar validity?

Suggested answers

1. A fixed offset identifies a proposed instant, but its local fields and offset may disagree with London's rules. Convert through UTC back into London and compare both fields and offset.
2. The fold genuinely contains two instants with the same wall label, each with a valid offset. A gap contains no instant with that label. Neither proposed gap interpretation returns to the original label.
3. They denote the same zero offset and are duplicates when the remaining version-key fields match. A later availability instant for the same observation creates a distinct revision.
4. No. The function returns only after validating the whole batch. A source iterator has still been consumed through the failing row, so callers needing replay must retain or reconstruct their input.
5. No. It checks internal structure and clock consistency. Access rights, truthful availability, historical vintages and calendar membership need independent evidence and additional validation.